

Normalization

DB — course 5

Normal forms

1NF

- A table is in the 1NF if all its attributes contain basic values (non-decomposable) meaning that each tuple **must not have group or repetitive data**.
- Hierarchical and network structures become elementary attributes tables.
- A table in NF1 still has a number of potential update anomalies due to *incomplete functional dependencies*.
- Each repetitive structure generates (by decomposition) a new table and group-level attributes are removed, leaving only the elementary ones.

E.g.: - group and repetitive attributes

- STUDENT (id, *name*, *addresses*, group, year) =>
- STUDENT (id, name, surname, group, year)
- ADDRESS (id, student_id, street, no, city, county)

2NF

- A table is in NF2 if and only if it is in NF1 and each non-key attribute of the table is **fully functionally dependent on the key**.
- An attribute B of a table functionally depends on attribute A of the same table if each value of A corresponds to one value of B, which is associated in the table.
- An attribute B is fully functionally dependent on a set of attributes A in the same table, if B is functionally dependent on the entire ensemble (not just an attribute of the ensemble).
- A table in NF2 still has a number of update anomalies due to any *transitive dependencies*.
- The removal of the incomplete dependencies is done by decomposing the original table in two tables both containing the intermediate (B) attribute.

E.g.: - fully functionally dependence

- STUDENT (name, year, group_no, no_stud_group, group_secretary, birth_date) =>
- STUDENT(name, group_no, birth_date)
- GROUP(group_no, year, no_stud, secretary)

3NF

- A table is normalized NF3 if and only if it is in NF2 and each non-key attribute **depends in a non-transitive way** on the table key.
- In a table T, consider A, B, C three attributes with A the key. If B depends on A ($A \rightarrow B$) and C depends on B ($B \rightarrow C$), then C depends on A by transitivity.
- The removal of transitive dependency is performed by decomposing the original table in two tables both containing the intermediate (B) attribute.
- A table in NF3 still has a number of update anomalies due any *multi-value dependencies*.
- A more refined definition for NF3 was given by BCNF (Boyce Codd Normal Form): A table is in BCNF if each *determinant is a key candidate*.
- The determinant is an elementary or compound attribute to which other attributes are fully functionally dependent.

E.g.: - non-transitive dependence

- STUDENT (id, name, surname, faculty_name, city) =>



- STUDENT (id, name, surname, faculty)
- FACULTY (faculty_name, city)

4NF

- A table is NF4 if and only if is NF3 and is not containing two or more **multi-value dependencies**.
- In a table T, consider A, B, C three attributes. The table T maintains the multi-value dependence on A ($\twoheadrightarrow$) if and only if the set of B values corresponding to a given pair (A, C) depends only on the one value of A and is independent of the values of C.

E.g.: - multi-value dependence

- STUDENT (name, language, subject) =>
- STUDENT (name, language)
- SUBJECTS (name, subject)

NAME	LANGUAGE	SUBJECT
Ionescu	Romanian	History
Popescu	Romanian	History
Popescu	Romanian	Mathematics
Popescu	French	History
Popescu	French	Mathematics

NAME	LANGUAGE
Ionescu	Romanian
Popescu	Romanian
Popescu	French

NAME	SUBJECT
Ionescu	History
Popescu	History
Popescu	Mathematics

5NF

- A table is NF5 if and only if it is NF4 and **each join dependence is generated by a key candidate of the table.**
- The table T (A, B, C) maintains the join dependence (AB, AC) if and only if T maintains the multi-value dependency $A \twoheadrightarrow B$ or $A \twoheadrightarrow C$.
- Multi-value dependency is a particular case of join dependency.
- Functional dependence is a particular case of multi-value dependency.
- The 4th and 5th NF are rarely used because they can generate redundancy.

E.g.: - join dependence

- ACTIVITY (student, subject, professor) =>
- ACTIVITY1 (student, subject)
- ACTIVITY2 (subject, professor)
- ACTIVITY3 (student, professor)

STUDENT

Ionescu

Ionescu

Ionescu

Ionescu

Popescu

SUBJECT

DB

DB

DBMS

DBMS

DB

PROFESSOR

Belciu

Diaconita

Belciu

Diaconita

Belciu

STUDENT

Ionescu

Ionescu

Popescu

PROFESSOR

Belciu

Diaconita

Belciu

SUBJECT

DB

DB

DBMS

DBMS

PROFESSOR

Belciu

Diaconita

Belciu

Diaconita

STUDENT

Ionescu

Ionescu

Popescu

SUBJECT

DB

DBMS

DB

Popescu studies DB. Diaconita teaches DB.
Popescu doesn't have Diaconita as a teacher.

Normalization example

Using the normalization technique, optimize the database schema for the HR department in a company

job_id, job_title, job_description, employee_id, name,
hire_date, salary, department_id, department_name,
end_date

NF1

- Identify repeating and group attributes.
- Remove these repeating attributes to a new table together with a copy of the key
- Split the group attributes

NF1

- Jobs (job_id, job_title, job_description)
- Employees (job_id, employee_id, first_name, last_name, hire_date, salary, department_id, department_name, end_date)

NF2

- Is each non-key attribute dependent on one part of the key?
- If yes, remove the attribute to a new table with a copy of the part of the key it is dependent upon.

NF2

- Jobs (job_id, job_title, job_description)
- Jobs_history (job_id, employee_id, end_date)
- Employees (employee_id, first_name, last_name, hire_date, salary, department_id, department_name)

NF3

- If a non-key attribute is more dependent on another non-key attribute than the table key (transitively-dependent on the key):
- Move the dependent attribute, together with a copy of the non-key attribute upon which it is dependent, to a new table.
- Leave the non-key attribute, upon which it is dependent, in the original table and mark it a foreign key

NF3

- Jobs (job_id, job_title, job_description)
- Jobs_history (job_id, employee_id, end_date)
- Employees (employee_id, first_name, last_name, hire_date, salary, department_id)
- Departments (department_id, department_name)

Relational Algebra and Calculus

- According to the relational model relational algebra and calculus operators are used for manipulating data.
- In order to implement these operators, there are specific DML commands in RDBMS. These commands are used for retrieval operations (queries).
- Considering the technique used for manipulation, the DML can be based on:
 - ☐ relational calculus (QUEL in Ingres, ALPHA proposed by Codd);
 - ☐ relational algebra (ISBL, RDMS);
 - ☐ transformation (SQL, SQUARE);
 - ☐ graphics (QBE, QBF).

Relational calculus

- Relational calculus is based on first-order predicate calculus (a domain of logic) and was proposed by E.F. Codd.
- The predicate is a relationship established between certain elements that can be confirmed or not.
- First-order predicate is a relation that has as arguments variables that are not predicates.
- The variable can be a tuple (the values are from a tuple of a table) or a domain (values are from a domain of a table).
- The quantifiers (operators) are symbols used in relational calculus: existential $\exists$, universal $\forall$.
- The basic construction in relational calculus is the tuple or domain relational expression (depending on the type of used variable).

- The relational calculus expression contains:
 - ☐ performed operation;
 - ☐ variables (tuple or domain);
 - ☐ conditions (comparison, existence);
 - ☐ defined formulas (operands: constants, variables, functions, predicates; operators);
 - ☐ quantifiers.

Relational Algebra

- Relational Algebra is a collection of formal operations applied to tables (relations) and was defined by E.F. Codd.
- The operations are applied to relational algebra expressions that are retrieval requests.
- Relational algebra expressions are composed of relational operators and operands.
- The operands are always tables (one or more).
- The result of evaluating a relational expression consists of a single table.
- The relational algebra has more or equal retrieval power than the relational calculus. An expression of relational calculus can be transformed into an equivalent relational algebra expression and vice versa.
- Relational algebra operators can be grouped into two categories: set and special relational operators.

1. Set operators

(We consider that R_1 , R_2 , R_3 are relations (tables))

- UNION: $R_3 = R_1 \cup R_2$, where R_3 will contain distinct tuples of R_1 and R_2 taken only once.
- MINUS: $R_3 = R_1 \setminus R_2$, where R_3 will contain tuples from R_1 which are not found in R_2
- CARTESIAN PRODUCT (CROSS JOIN): $R_3 = R_1 \times R_2$, R_3 will contain tuples that are made of pairs (x_1x_2) , having $x_1 \in R_1$ and $x_2 \in R_2$.
- INTERSECT: $R_3 = R_1 \cap R_2$, R_3 will contain tuples which are the same in R_1 and in R_2 .

2. Special relational operators

SELECTION: sub-table R2 is obtained from R1 table and will contain a subset of the original tuples in R1 that satisfy a predicate (condition).

- The number of attributes in R2 = number of attributes in R1
- The number of tuples in R2 $\leq$ number of tuples in R1.
- -> WHERE clause **ex: select * from students where extract(month from birth_date)=2;**

PROJECTION: sub-table R2 is obtained from R1 table and will contain a subset of the original attributes of R1 and no duplicate tuples.

- The number of attributes in R2 < number of attributes in R1
- The number of tuples in R2 ≤ number of tuples in R1
- Students(id, name, birth_date, group)

11, ion, 22.02.1990, 1000

12, ion, 22.02.1990, 1009

13, ion, 23.02.1990, 1009

- **Ex: select DISTINCT name, birth_date from students;**

ion, 22.02.1990

ion, 23.02.1990

- **Ex: select DISTINCT name from students;**

ion

- -> FIELDS clause + DISTINCT

JOIN: is a type of Cartesian product, which involves using a qualifier in order to compare the values of attributes in R1 and R2, and to obtain the result in R3.

- R1 and R2 must have one or more common attributes which have the same values.
- Types of joins:
 - equijoin (natural join – a sub-type)
 - outer join (left, right, full outer join)
 - semi-join (data is extracted from a table and the condition is put on another table)
 - self-join (for a self-referenced table)
 - non-equijoin (the condition is not “=”)

Equijoin – natural join

Ex: Students (**student_id**, student_name, birthdate)

Addresses (address_id, city, no, street, **student_id**)

- Select student_name, address_id from students NATURAL JOIN addresses

Ex: Students (**id**, student_name, birthdate)

Addresses (address_id, city, no, street, **student_id**)

- X natural join

Ex: Students (**student_id**, *name*, birthdate)

Addresses (address_id, *name*, city, no, street, **student_id**)

- X natural join

- Codd introduced six basic operators (union, difference, Cartesian product, selection, projection, join) and two derived operators (intersection and division).
- Other special operators were later introduced, like complementarity or splitting.
- Relational algebra is by definition non-procedural (descriptive), but relational calculus allows mixed searches (procedural / non-procedural).

Relational transformation

- The relational transformation is a domain of relational theory that has a retrieval power, which is equivalent to relational calculus or algebra.
- It is based on the transformation (mapping) of an attribute or group of attributes in a required attribute using relations. **The fundamental idea is that an expression of relational calculus can be transformed into an equivalent relational algebra expression and vice versa.**
- The result is a relation (table) that can be used in another transformation.

Graphics

- The graphics provide high interactivity when building requests and offers Wizard assistants.
- The user specifies the request by choosing or completing a structured graphic screen.
- All users in informatics that are specialists or not can use it.